

All or nothing: transactions



Make a money transfer happen completely, or not at all.

The big idea

Ada sends Bob 25.00. That's **three writes**: take it off Ada, add it to Bob, save a log. If the server crashes after the first, Ada's money is simply gone.

A **transaction** is a clerk who fills in the whole form in **pencil**, and only inks it once every line is right. One wrong line, and the whole form goes in the bin.

What your code receives

The tests create two accounts, then pass their **ids** and an amount in cents.

```
ada = Account.objects.create(owner="Ada", balance=10_000)    # 100.00
bob = Account.objects.create(owner="Bob", balance=500)      # 5.00

transfer(ada.id, bob.id, 2_500)    # tests 1-2: Ada 7_500, Bob 3_000, returns the log

# tests 3-7 must fail AND change nothing:
transfer(ada.id, bob.id, 12.5)     # ValueError (also 0, -50, "100")
transfer(ada.id, ada.id, 100)      # ValueError: same account
transfer(bob.id, ada.id, 501)      # InsufficientFunds: Bob only has 500
transfer(ada.id, 999_999, 100)     # Account.DoesNotExist
# test 7: saving the log fails with RuntimeError("disk full")
```

New words

transaction

A group of database writes kept all together, or not at all.

atomic

"Can't be split": the whole group happens, or none of it.

commit

Make the writes permanent, when the block finishes normally.

roll back

Undo every write in the group, when an exception leaves it.

lock

Mark a row "in use" so another request waits its turn.

cents

Money as whole numbers: **10_000** means 100.00.

Tool: your own exception

```
class TooCold(Exception):    # like InsufficientFunds in your file
    pass

def check_temperature(degrees):
    if degrees < 5:
        raise TooCold(f"{degrees} degrees is too cold to plant seeds")
    return "ok"

print(check_temperature(12))
check_temperature(2)
```

OUTPUT

```
ok
TooCold: 2 degrees is too cold to plant seeds
```

Tool: isinstance for the amount

Check the **type** first: `"100" <= 0` would crash with a `TypeError`.

```
for amount in [250, 0, 12.5, "100"]:  
    bad = not isinstance(amount, int) or amount <= 0  
    print(repr(amount), "rejected" if bad else "ok")
```

OUTPUT

```
250 ok  
0 rejected  
12.5 rejected  
'100' rejected
```

Tool: transaction.atomic()

```
zoe = Account.objects.create(owner="Zoe", balance=300)
try:
    with transaction.atomic():          # without it, Zoe ends with 200
        zoe.balance -= 100
        zoe.save()
        raise RuntimeError("power cut")
except RuntimeError as error:
    print("Caught:", error)

zoe.refresh_from_db()    # re-read the balance from the database
print(zoe.balance)
```

OUTPUT

```
Caught: power cut
300
```

atomic() vs try / except

with transaction.atomic():

- **Undoes** every write in the block
- Lets the error carry on out
- The caller still sees what went wrong
- Use it around all of **transfer**

try: ... except: ...

- **Catches** the error and stops it
- Does **not** undo any saves
- Hides the error from the caller
- Don't use it in **transfer**

Tool: select_for_update

Locks the row until the transaction ends, so two transfers can't both spend the same money.

```
with transaction.atomic():
    zoe = Account.objects.select_for_update().get(pk=zoe_id)
    zoe.balance -= 100
    zoe.save()
# the lock on Zoe's row is released here
```

Your task, step by step

- 1 If `amount` isn't an `int`, or is 0 or less, raise `ValueError` → **test 3**
- 2 If `source_id == destination_id`, raise `ValueError` → **test 4**
- 3 Move all the database code inside `with transaction.atomic():` → **test 7**
- 4 Fetch both accounts first with `select_for_update().get(pk=...)` → **test 6**
- 5 If `source.balance < amount`, raise `InsufficientFunds` → **test 5**
- 6 Subtract, add, and `.save()` both accounts → **test 1**
- 7 Keep `return TransferLog.objects.create(...)` inside the block → **tests 2, 7**

Common mistakes

- Leaving some `save()` lines outside the `with` block (not indented)
- Wrapping it in `try / except` and hiding the error
- Using `int(amount): 12.5` and `"100"` must be rejected
- Checking the balance **after** subtracting
- Using `TransferLog(...).save()`: test 7 needs `objects.create`

Where you'll use this

A shop places an order and reduces the stock. If the stock update fails, the order must not exist either, or you sell things you don't have.

- Any action writing to several rows needs to be all-or-nothing
- In a Django view, wrap that work in `transaction.atomic()`
- `ATOMIC_REQUESTS` makes every request one transaction

Check yourself

1. What happens to earlier saves when an exception leaves an `atomic` block?
2. Why check `isinstance(amount, int)` before `amount <= 0`?
3. Why not catch the error inside `transfer`?
4. What problem does `select_for_update()` prevent?

Keep learning

[ACID Databases – Atomicity, Consistency, Isolation & Durability Explained](#)

freeCodeCamp · article · beginner. Start here: restaurant analogies that make 'all or nothing' (atomicity) and isolation easy to picture.

[PostgreSQL tutorial: Transactions](#)

PostgreSQL docs · tutorial · beginner. A short bank-transfer example in plain SQL showing BEGIN, COMMIT and rollback, the same scenario as this exercise.

[Errors and Exceptions: user-defined exceptions](#)

Python docs · docs · beginner. How to make your own exception class like InsufficientFunds, which triggers the rollback when raised.

[Database transactions: controlling transactions explicitly](#)

Django docs · docs · intermediate. Official reference for transaction.atomic() and how an exception inside the with block undoes every change.

[QuerySet API reference: select for update](#)

Django docs · docs · intermediate. Explains row locking so two transfers cannot change the same account at the same time.